

GenAI #1: Grok

Accessed through: Grok website

GenAI #2: Microsoft Copilot

Accessed through: Microsoft Copilot website

Prompt used:

Write a C++ program for a CEO email priority system using objects and classes. Use a MaxHeap with a list-based implementation and don't use any built-in heap or priority queue. Build the MaxHeap from scratch.

Each email has a sender category, subject, and date.

The priority from highest to lowest is:

Boss

Subordinate

Peer

ImportantPerson

OtherPerson

If two emails have the same sender category, the newer email should come first. The date format is MM-DD-YYYY.

The program reads commands from a test file. An EMAIL command is all on ONE LINE and looks exactly like this:

EMAIL Peer,Can you help me on this?,12-01-2024

After "EMAIL " the sender, subject, and date are separated by commas. The subject can have spaces but will not have commas.

The commands are:

EMAIL - adds the email to the MaxHeap.

NEXT - shows the highest priority email without removing it.

READ - removes the highest priority email without displaying it.

COUNT - shows the number of unread emails.

The program also needs to work if NEXT or READ happens when there are no emails. Two NEXT commands in a row should show the same email. Two READ commands in a row should remove the two highest priority emails.

The input file can have any number and order of these commands.

Keep the C++ code simple and student level. Use objects/classes and make sure the MaxHeap is actually implemented as a MaxHeap, not just an unsorted list that searches for the highest priority email.

Copilot Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
using namespace std;
class Email {
public:
    string sender;
    string subject;
    string date;
    Email() {}
    Email(string s, string sub, string d) {
        sender = s;
        subject = sub;
        date = d;
    }
};
class MaxHeap {
private:
    vector<Email> heap;
    int getPriority(string sender) {
        if (sender == "Boss")
            return 5;
        else if (sender == "Subordinate")
            return 4;
```

```

    else if (sender == "Peer")
        return 3;
    else if (sender == "ImportantPerson")
        return 2;
    else
        return 1;
}

int convertDate(string date) {
    string month = date.substr(0, 2);
    string day = date.substr(3, 2);
    string year = date.substr(6, 4);
    return stoi(year + month + day);
}

bool higherPriority(Email a, Email b) {
    int p1 = getPriority(a.sender);
    int p2 = getPriority(b.sender);
    if (p1 > p2)
        return true;
    if (p1 < p2)
        return false;
    return convertDate(a.date) > convertDate(b.date);
}

void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (higherPriority(heap[index], heap[parent])) {
            swap(heap[index], heap[parent]);
            index = parent;
        }
        else {
            break;
        }
    }
}

void heapifyDown(int index) {
    int size = heap.size();
    while (true) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;
        if (left < size &&
            higherPriority(heap[left], heap[largest])) {
            largest = left;
        }
        if (right < size &&
            higherPriority(heap[right], heap[largest])) {

```

```

        largest = right;
    }
    if (largest != index) {
        swap(heap[index], heap[largest]);
        index = largest;
    }
    else {
        break;
    }
}
}
public:
    void insert(Email email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }
    bool isEmpty() {
        return heap.empty();
    }
    int count() {
        return heap.size();
    }
    Email getMax() {
        return heap[0];
    }
    void removeMax() {
        if (heap.empty())
            return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty())
            heapifyDown(0);
    }
};

int main() {
    string fileName;
    cout << "Enter file name: ";
    cin >> fileName;
    ifstream inputFile(fileName);
    if (!inputFile) {
        cout << "Could not open file." << endl;
        return 1;
    }
    MaxHeap emails;
    string line;
    while (getline(inputFile, line)) {

```

```

if (line.substr(0, 5) == "EMAIL") {
    string data = line.substr(6);
    int comma1 = data.find(',');
    int comma2 = data.find(',', comma1 + 1);
    string sender = data.substr(0, comma1);
    string subject =
        data.substr(comma1 + 1,
            comma2 - comma1 - 1);
    string date =
        data.substr(comma2 + 1);
    Email email(sender, subject, date);
    emails.insert(email);
}
else if (line == "NEXT") {
    if (emails.isEmpty()) {
        cout << "No unread emails." << endl;
    }
    else {
        Email next = emails.getMax();
        cout << "Next email:" << endl;
        cout << "Sender: " << next.sender << endl;
        cout << "Subject: " << next.subject << endl;
        cout << "Date: " << next.date << endl;
    }
}
else if (line == "READ") {
    if (emails.isEmpty()) {
        cout << "No unread emails." << endl;
    }
    else {
        emails.removeMax();
    }
}
else if (line == "COUNT") {
    cout << "There are "
        << emails.count()
        << " emails to read."
        << endl;
}
}
inputFile.close();
return 0;
}

```

Grok Code:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <vector>
```

```
#include <sstream>
```

```
using namespace std;
```

```
// Convert MM-DD-YYYY to integer YYYYMMDD for comparison
```

```
int dateToInt(const string& date) {
```

```
    int month = stoi(date.substr(0, 2));
```

```
    int day   = stoi(date.substr(3, 2));
```

```
    int year  = stoi(date.substr(6, 4));
```

```
    return year * 10000 + month * 100 + day;
```

```
}
```

```
// Lower number = higher priority
```

```
int getCategoryRank(const string& category) {
```

```
    if (category == "Boss")        return 0;
```

```
    if (category == "Subordinate") return 1;
```

```
    if (category == "Peer")        return 2;
```

```
    if (category == "ImportantPerson") return 3;
```

```
    if (category == "OtherPerson") return 4;
```

```
    return 5;
```

```
}
```

```
class Email {
```

private:

string sender;

string subject;

string date;

int dateValue;

int rank;

public:

Email() {

sender = "";

subject = "";

date = "";

dateValue = 0;

rank = 5;

}

Email(const string& s, const string& sub, const string& d) {

sender = s;

subject = sub;

date = d;

dateValue = dateToInt(d);

rank = getCategoryRank(s);

}

string getSender() const { return sender; }

string getSubject() const { return subject; }

string getDate() const { return date; }

```

// true if this email has strictly higher priority than other
bool hasHigherPriorityThan(const Email& other) const {
    if (rank != other.rank)
        return rank < other.rank;    // smaller rank is better
    return dateValue > other.dateValue; // newer date is better
}

```

```

void display() const {
    cout << "Next email:" << endl;
    cout << "Sender: " << sender << endl;
    cout << "Subject: " << subject << endl;
    cout << "Date: " << date << endl;
}
};

```

```

// Max-Heap built from scratch using a vector (list) as storage.
// Index 0 is unused so parent/child formulas are simple:
// parent(i) = i/2, left(i) = 2*i, right(i) = 2*i+1
class MaxHeap {
private:
    vector<Email> data; // data[0] is a dummy, real elements start at 1
    int heapSize;

    void swap(int i, int j) {
        Email temp = data[i];
        data[i] = data[j];

```

```
data[j] = temp;
```

```
}
```

```
// Move a newly inserted element up to restore heap property
```

```
void bubbleUp(int index) {
```

```
    while (index > 1) {
```

```
        int parent = index / 2;
```

```
        if (data[index].hasHigherPriorityThan(data[parent])) {
```

```
            swap(index, parent);
```

```
            index = parent;
```

```
        } else {
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
// Move the root down after removal to restore heap property
```

```
void bubbleDown(int index) {
```

```
    while (true) {
```

```
        int left = 2 * index;
```

```
        int right = 2 * index + 1;
```

```
        int largest = index;
```

```
        if (left <= heapSize && data[left].hasHigherPriorityThan(data[largest]))
```

```
            largest = left;
```

```
        if (right <= heapSize && data[right].hasHigherPriorityThan(data[largest]))
```

```
            largest = right;
```

```
        if (largest != index) {  
            swap(index, largest);  
            index = largest;  
        } else {  
            break;  
        }  
    }  
}
```

public:

```
    MaxHeap() {  
        data.push_back(Email()); // dummy at index 0  
        heapSize = 0;  
    }
```

```
    void insert(const Email& e) {  
        // grow the vector if needed  
        if (heapSize + 1 >= (int)data.size())  
            data.push_back(Email());  
        heapSize++;  
        data[heapSize] = e;  
        bubbleUp(heapSize);  
    }
```

```
    // Look at the highest-priority email without removing it  
    bool peek(Email& result) const {
```

```
    if (heapSize == 0) return false;
```

```
    result = data[1];
```

```
    return true;
```

```
}
```

```
// Remove the highest-priority email
```

```
bool removeMax() {
```

```
    if (heapSize == 0) return false;
```

```
    // move last element to root and shrink
```

```
    data[1] = data[heapSize];
```

```
    heapSize--;
```

```
    if (heapSize > 0)
```

```
        bubbleDown(1);
```

```
    return true;
```

```
}
```

```
int size() const {
```

```
    return heapSize;
```

```
}
```

```
bool isEmpty() const {
```

```
    return heapSize == 0;
```

```
}
```

```
};
```

```
// Parse one EMAIL line: "EMAIL Sender,Subject with spaces,MM-DD-YYYY"
```

```
bool parseEmailLine(const string& line, string& sender, string& subject, string& date) {
```

```

// skip the word "EMAIL "
size_t pos = line.find(' ');
if (pos == string::npos) return false;
string rest = line.substr(pos + 1);

// first comma separates sender
size_t c1 = rest.find(',');
if (c1 == string::npos) return false;
sender = rest.substr(0, c1);

// second comma separates subject from date
size_t c2 = rest.find(',', c1 + 1);
if (c2 == string::npos) return false;
subject = rest.substr(c1 + 1, c2 - c1 - 1);
date = rest.substr(c2 + 1);

// trim possible leading/trailing spaces on date
while (!date.empty() && date[0] == ' ') date.erase(0, 1);
while (!date.empty() && date.back() == ' ') date.pop_back();

return true;
}

int main() {
    ifstream infile("test.txt");
    if (!infile) {
        cout << "Error: could not open test.txt" << endl;
    }
}

```

```
return 1;
```

```
}
```

```
MaxHeap heap;
```

```
string line;
```

```
while (getline(infile, line)) {
```

```
    // skip empty lines
```

```
    if (line.empty()) continue;
```

```
    // check the command
```

```
    if (line.compare(0, 5, "EMAIL") == 0) {
```

```
        string sender, subject, date;
```

```
        if (parseEmailLine(line, sender, subject, date)) {
```

```
            Email e(sender, subject, date);
```

```
            heap.insert(e);
```

```
        }
```

```
    }
```

```
    else if (line == "NEXT") {
```

```
        Email top;
```

```
        if (heap.peek(top)) {
```

```
            top.display();
```

```
        }
```

```
        // empty heap: do nothing
```

```
    }
```

```
    else if (line == "READ") {
```

```
        heap.removeMax(); // does nothing if empty
```

```

    }
    else if (line == "COUNT") {
        cout << "There are " << heap.size() << " emails to read." << endl;
    }
}

infile.close();
return 0;
}

```

Correctness

Both codes follow the right email priority and put newer emails first when they have the same sender type.

Both also read the EMAIL line correctly and split the sender, subject, and date.

Grok does some extra checks when reading the input. Copilot is more simple and expects the test file to be formatted correctly.

Execution Time

Both use a MaxHeap.

- EMAIL: $O(\log n)$
- NEXT: $O(1)$
- READ: $O(\log n)$
- COUNT: $O(1)$

Grok saves the rank and converted date inside each email. Copilot calculates them when it needs to compare emails.

Space Complexity

Both use $O(n)$ space because they store all the emails in a vector.

Maintainability

Copilot is shorter and easier to understand.

Grok is more detailed and has more methods and variables, which makes the code longer.

Both use separate Email and MaxHeap classes.

Program Selected

I picked Copilot as my base code.

I picked it because both programs worked and had similar time and space complexity, but Copilot's code was shorter and simpler. It was easier to understand, maintain, and make changes to.

Improvements

I tested the code with different commands and empty cases to improve correctness. I kept the MaxHeap and vector because they already have good time and space complexity, and I added better comments to make the code easier to understand and maintain.